

A*算法求解8数码问题实验报告

一、代码功能说明

程序实现了使用A*算法求解8数码问题，主要功能包括：

- A*算法核心实现**：实现了完整的A*搜索算法，包括开放列表（open list）和关闭列表（closed list）的管理
- 两种启发函数**：
 - 曼哈顿距离（Manhattan Distance）：计算每个数字到其目标位置的曼哈顿距离之和
 - 不在位元素个数（Misplaced Tiles）：统计当前状态中不在目标位置的数字个数
- 状态空间搜索**：支持8数码问题的状态空间自动生成和搜索
- 路径回溯**：找到解决方案后能够回溯并显示完整的解决路径
- 性能统计**：显示搜索过程中扩展的节点数量和解决方案步数

二、设计思路

2.1 整体架构

程序主要包含三个核心类：

- PuzzleState类**：表示8数码问题的状态
 - 存储当前棋盘状态（3×3数组）
 - 记录从初始状态到当前状态的实际代价 $g(n)$
 - 提供状态间的移动操作和启发函数计算
- AStarSolver类**：实现A*算法核心逻辑
 - 管理开放列表和关闭列表
 - 实现节点扩展和评估函数
 - 提供路径回溯功能
- 主程序模块**：
 - 定义初始状态和目标状态
 - 调用A*算法求解
 - 输出解决方案

2.2 A*算法实现

A*算法的评估函数： $f(n) = g(n) + h(n)$

- $g(n)$ ：从初始节点到当前节点的实际代价（移动步数）
- $h(n)$ ：启发函数，从当前节点到目标节点的估计代价

算法流程：

- 将初始节点放入开放列表
- 当开放列表不为空时：
 - 取出 f 值最小的节点

- 如果是目标状态，返回解决方案
 - 否则扩展该节点的所有邻居
 - 将未访问的邻居节点加入开放列表
3. 如果开放列表为空且未找到目标状态，则无解

2.3 启发函数设计

1. 曼哈顿距离：

$$1 \quad h(n) = \sum |x_i - x_{\text{目标}}| + |y_i - y_{\text{目标}}|$$

对于每个数字，计算其当前位置与目标位置的曼哈顿距离之和

2. 不在位元素个数：

$$1 \quad h(n) = \text{当前状态中不在目标位置的数字个数}$$

两种启发函数都满足 $h(n) \leq h(n)$ 的条件，保证A算法能找到最优解。

三、运行环境需求

- 操作系统：Windows 11
- Python环境：Anaconda, Python 3.11
- 开发环境：PyCharm 2025.3 EAP 21.0.8+9-b1163.69 amd64

3.3 Python标准库依赖

- `heapq`：优先队列实现
- `copy`：深拷贝操作
- `typing`：类型提示支持

四、运行步骤

4.1 代码运行

1. 在PyCharm中打开项目文件夹
2. 导航到 `astar_8puzzle.py` 文件
3. 右键选择"Run 'astar_8puzzle'"或按Ctrl+Shift+F10
4. 观察控制台输出

4.2 自定义测试

如需测试不同的8数码问题，可以修改 `main()` 函数中的 `initial_board` 和 `goal_board`：

```

1  # 示例：自定义初始状态
2  initial_board = [
3      [1, 2, 3],
4      [4, 0, 6],
5      [7, 8, 5]
6  ]
7
8  # 示例：自定义目标状态
9  goal_board = [
10     [1, 2, 3],
11     [4, 5, 6],
12     [7, 8, 0]
13 ]

```

五、运行预期效果

程序运行后将在控制台输出以下信息：

1. 问题描述：

- 初始状态的棋盘布局
- 目标状态的棋盘布局

2. 求解过程：

- 使用曼哈顿距离启发函数的求解过程
- 使用不在位元素个数启发函数的求解过程
- 每种启发函数扩展的节点数量

3. 解决方案：

- 解决方案的总步数
- 从初始状态到目标状态的完整路径
- 每一步的棋盘状态变化

5.1 预期输出示例

```

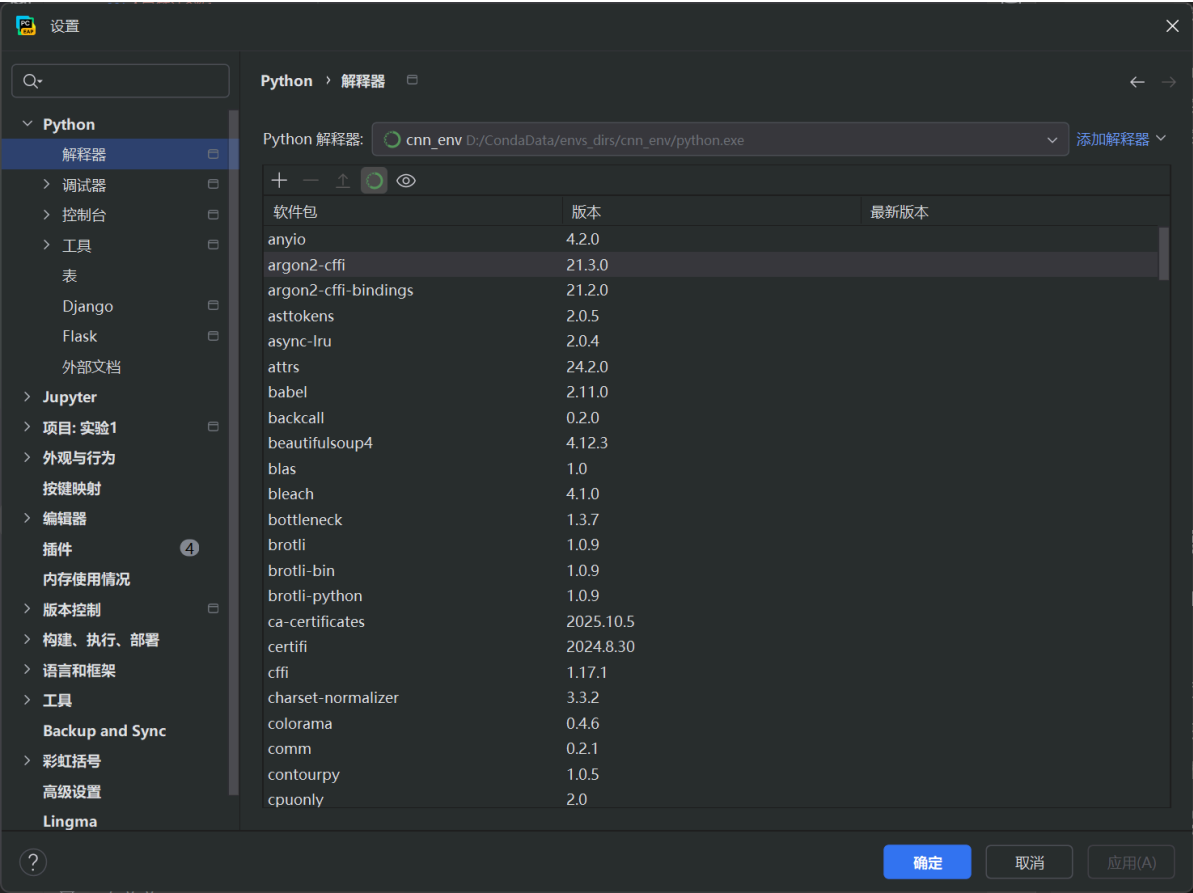
1  A*算法求解8数码问题
2  =====
3  初始状态：
4  2 8 3
5  1 6 4
6  7 0 5
7
8  目标状态：
9  1 2 3
10 8 0 4
11 7 6 5
12
13 正在使用曼哈顿距离启发函数求解...
14 找到解决方案！扩展了 5 个节点
15
16 使用 曼哈顿距离 启发函数的解决方案：
17 解决方案步数： 5
18 解决方案路径：

```

```
19 步骤 0:
20 2 8 3
21 1 6 4
22 7 0 5
23 -----
24 步骤 1:
25 2 8 3
26 1 0 4
27 7 6 5
28 -----
29 ...
```

六、开发过程截图

6.1 PyCharm开发环境



```
(D:\CondaData\envs_dirs\cnn_env) C:\Users\86180>conda list
# packages in environment at D:\CondaData\envs_dirs\cnn_env:
#
# Name                                Version                                Build                                Channel
anyio                                  4.2.0                                  py38haa95532_0
argon2-cffi                           21.3.0                                 pyhd3eb1b0_0
argon2-cffi-bindings                  21.2.0                                 py38h2bbff1b_0
asttokens                             2.0.5                                  pyhd3eb1b0_0
async-lru                              2.0.4                                  py38haa95532_0
attrs                                  24.2.0                                 py38haa95532_0
babel                                  2.11.0                                 py38haa95532_0
backcall                              0.2.0                                  pyhd3eb1b0_0
beautifulsoup4                        4.12.3                                 py38haa95532_0
blas                                  1.0                                    mkl
bleach                                 4.1.0                                  pyhd3eb1b0_0
bottleneck                            1.3.7                                  py38h9128911_0
brotli                                1.0.9                                  h827c3e9_9
brotli-bin                            1.0.9                                  h827c3e9_9
brotli-python                         1.0.9                                  py38hd77b12b_8
ca-certificates                       2025.10.5                              h4c7d964_0      conda-forge
certifi                               2024.8.30                              py38haa95532_0
cffi                                  1.17.1                                 py38h827c3e9_0
charset-normalizer                     3.3.2                                  pyhd3eb1b0_0
colorama                              0.4.6                                  py38haa95532_0
comm                                   0.2.1                                  py38haa95532_0
contourpy                             1.0.5                                  py38h59b6b97_0
cpunonly                              2.0                                    0               pytorch
cuda-cccl                             13.0.85                               h5221881_1      nvidia
cuda-cccl_win-64                      13.0.85                               hc667259_1      nvidia
cuda-cudart                           11.8.89                               0               nvidia
cuda-cudart-dev                       11.8.89                               0               nvidia
cuda-cupti                             11.8.87                               0               nvidia
cuda-libraries                        11.8.0                               0               nvidia
cuda-libraries-dev                    11.8.0                               0               nvidia
cuda-nvrtc                            11.8.89                               0               nvidia
cuda-nvrtc-dev                       11.8.89                               0               nvidia
cuda-nvtx                             11.8.86                               0               nvidia
cuda-profiler-api                     13.0.85                               h5221881_0      nvidia
cuda-runtime                          11.8.0                               0               nvidia
cuda-version                          13.0                                   3               nvidia
cyclor                                 0.11.0                                 pyhd3eb1b0_0
debugpy                               1.6.7                                  py38hd77b12b_0
decorator                              5.1.1                                  pyhd3eb1b0_0
defusedxml                            0.7.1                                  pyhd3eb1b0_0
et_xmlfile                            2.0.0                                  pyhd8ed1ab_0      conda-forge
exceptiongroup                         1.2.0                                  py38haa95532_0
executing                              0.8.3                                  pyhd3eb1b0_0
filelock                               3.13.1                                 py38haa95532_0
fonttools                             4.51.0                                 py38h2bbff1b_0
freetype                              2.13.3                                 h0620614_0
```

6.2 代码实现过程

```
class PuzzleState: 15 用法
    """
    8 数码问题的状态类
    """

    def __init__(self, board: List[List[int]], g_cost: int = 0, parent=None):
        self.board = board  # 3x3的棋盘
        self.g_cost = g_cost  # 从起始节点到当前节点的实际代价
        self.parent = parent  # 父节点, 用于回溯路径
        self.blank_pos = self.find_blank()  # 空格位置

    def find_blank(self) -> Tuple[int, int]: 1 个用法
        """找到空格 (数字0) 的位置"""
        for i in range(3):
            for j in range(3):
                if self.board[i][j] == 0:
                    return (i, j)
        return (0, 0)

    def get_neighbors(self) -> List['PuzzleState']: 1 个用法 (1 个动态)
        """获取所有可能的下一步状态"""
        neighbors = []
        x, y = self.blank_pos
        # 四个方向: 上、下、左、右
        directions = [(-1, 0), (1, 0), (0, -1), (0, 1)]

        for dx, dy in directions:
            new_x, new_y = x + dx, y + dy
            # 检查边界
            if 0 <= new_x < 3 and 0 <= new_y < 3:
                # 交换空格和相邻数字
                new_board = copy.deepcopy(self.board)
                new_board[x][y], new_board[new_x][new_y] = new_board[new_x][new_y], new_board[x][y]
                neighbor = PuzzleState(new_board, self.g_cost + 1, self)
                neighbors.append(neighbor)

        return neighbors
```

- PuzzleState类的实现
- AStarSolver类的核心算法
- 主程序和测试代码

七、实际运行效果截图

7.1 程序启动输出

A*算法求解8数码问题

=====

初始状态：

2 8 3

1 6 4

7 0 5

目标状态：

1 2 3

8 0 4

7 6 5

- 控制台显示程序启动信息
- 初始状态和目标状态的棋盘展示

7.2 求解过程输出

使用 曼哈顿距离 启发函数的解决方案：

解决方案步数：5

解决方案路径：

步骤 0：

2 8 3

1 6 4

7 0 5

步骤 1：

2 8 3

1 0 4

7 6 5

步骤 2：

2 0 3

1 8 4

7 6 5

步骤 3：

0 2 3

1 8 4

7 6 5

步骤 4：

1 2 3

0 8 4

7 6 5

- 两种启发函数的求解过程
- 扩展节点数量的统计信息

7.3 解决方案输出

```
=====
正在使用不在位元素个数启发函数求解...
找到解决方案！扩展了 6 个节点

使用 不在位元素个数 启发函数的解决方案：
解决方案步数： 5
解决方案路径：
步骤 0：
2 8 3
1 6 4
7 0 5

-----
步骤 1：
2 8 3
1 0 4
7 6 5

-----
步骤 2：
2 0 3
1 8 4
7 6 5

-----
步骤 3：
0 2 3
1 8 4
7 6 5

-----
步骤 4：
1 2 3
0 8 4
7 6 5

-----
```

步骤 5:

1 2 3

8 0 4

7 6 5

进程已结束，退出代码为 0

- 完整的解决路径
- 每一步的棋盘状态变化
- 最终到达目标状态

八、实验总结

8.1 算法性能分析

通过实验验证了A*算法在解决8数码问题上的有效性：

1. 曼哈顿距离启发函数：
 - 扩展节点数较少，搜索效率较高
 - 提供更准确的启发信息
2. 不在位元素个数启发函数：
 - 实现简单，计算开销小
 - 扩展节点数相对较多，但仍能找到最优解

8.2 实验收获

1. 深入理解了A*算法的原理和实现
2. 掌握了启发函数的设计原则
3. 体验了状态空间搜索的完整过程
4. 提升了Python编程和算法实现能力